Software Requirements Specification (SRS)

Project: Hardware Store Management System

Version: 1.3

Date: April 28, 2026

Author: Group 15

1. Introduction

1.1 Purpose

This document describes the requirements for the **Hardware Store Management System (HSMS)**. The system is a web-based full-stack application developed using:

- Frontend: React.js
- Backend: ASP.NET Web API
- Database: MySQL
- Deployment: Azure App Service

The purpose of this system is to automate and manage hardware store operations including inventory management, sales processing, supplier management, reporting, and user management.

This SRS is intended for:

- The development team (project group members)
- The module instructors / project evaluators
- Project supervisor
- Stakeholders
- QA engineers

1.2 Scope

The Hardware Store Management System will:

- Manage products (tools, materials, equipment)
- Track stock levels
- Manage suppliers
- Process sales using a cart-based transaction system with real-time stock validation and automatic invoice generation
- Generate dynamic reports (daily, monthly, low stock) with export functionality (CSV/PDF)
- Support multiple user roles (Admin, Cashier, Manager)

The system will improve:

- Accuracy of inventory tracking
- Sales efficiency
- Reporting automation
- Business decision-making

1.3 Definitions

Term	Description
HSMS	Hardware Store Management System
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous (Deployment/Delivery)
SKU	Stock Keeping Unit

2. Overall Description

2.1 Product Perspective

HSMS is a web-based system consisting of:

- React frontend (Client-side UI)
- ASP.NET Web API backend
- MySQL relational database
- Hosted on Azure

The system also includes:

- JWT-based authentication layer
- Role-based authorization at API and UI level
- CI/CD pipeline for automated deployment

High-Level Architecture

User → React Frontend → ASP.NET API → MySQL Database

2.2 Product Functions

The system shall:

1. Authenticate users using JWT-based login system with secure token generation and validation
2. Manage products (Add, update, delete, search)
3. Manage inventory levels
4. Process sales transactions using a cart-based system with product selection, quantity validation, and total calculation
5. Automatically generate invoices after successful sales transactions with print/download support
6. Manage suppliers
7. Generate dynamic reports including daily sales, monthly sales, and low stock reports with export functionality
8. Maintain system and transaction logs for auditing and debugging
9. Support role-based access control
10. Support CI/CD deployment

2.3 User Classes

User Type	Description	permissions
Admin	Full system access	Manage users, roles, products, suppliers, and reports
Manager	Manage reports, inventory, suppliers	View reports, manage inventory and suppliers, monitor sales
Cashier	Process sales only	Create sales transactions and generate invoices

2.4 Operating Environment

- Web browser (Chrome, Firefox, Edge)
- Windows/Linux server
- Azure App Service
- MySQL 8+

2.5 Design Constraints

- Must use React for frontend
- Must use ASP.NET + ADO.NET (direct SQL)
- Must use MySQL
- Must use GitHub
- Must implement CI/CD (GitHub Actions)
- Must implement unit, integration & load testing

3. System Features & Functional Requirements

3.1 User Authentication

Description

Users must log in to access the system.

Functional Requirements

FR1: The system shall allow users to log in using username and password.
FR2: The system shall validate credentials from the database.
FR3: The system shall implement role-based access control.
FR4: The system shall allow users to log out securely.
FR5: The system shall generate a JWT token upon successful login.
FR6: The system shall include user role information within the JWT token.
FR7: The system shall protect API endpoints using JWT authentication middleware.
FR8: The system shall restrict access to frontend routes based on user roles.

3.2 Product Management

Description

Admin/Manager can manage hardware products.

Functional Requirements

FR9: The system shall allow adding new products.
FR10: The system shall allow updating product details.
FR11: The system shall allow deleting products.
FR12: The system shall allow searching products.
FR13: The system shall store product name, SKU, price, quantity, category.

3.3 Inventory Management

FR14: The system shall update stock after each sale.
FR15: The system shall alert when stock is below threshold.
FR16: The system shall allow manual stock updates.
FR17: The system shall highlight low stock items based on a predefined threshold.
FR18: The system shall allow setting or modifying stock threshold values.

3.4 Sales Management

FR19: The system shall allow users to create sales transactions by selecting products and specifying quantities.

FR20: The system shall generate an invoice automatically after successful transaction completion.

FR21: The system shall calculate line item subtotals, overall totals, and applicable taxes.

FR22: The system shall store transaction history including items, quantities, totals, and timestamps.

FR23: The system shall prevent sales when requested quantity exceeds available stock.

FR24: The system shall update inventory only after successful transaction completion.

3.5 Supplier Management

FR25: The system shall allow adding suppliers.
FR26: The system shall allow updating supplier details.
FR27: The system shall delete suppliers.
FR28: The system shall maintain relationships between suppliers and products in the database.

3.6 Reporting

FR29: The system shall generate daily sales reports.
FR30: The system shall generate monthly sales reports.
FR31: The system shall generate low stock reports.
FR32: The system shall allow exporting reports (PDF/CSV).
FR33: The system shall allow filtering reports by date range.
FR34: The system shall generate low stock reports based on threshold values.
FR35: The system shall export reports in CSV format (minimum requirement).

3.7 User & Role Management

FR36: The system shall allow admin users to create new user accounts.

FR37: The system shall allow admin users to assign roles to users.

FR38: The system shall allow admin users to update user roles.

FR39: The system shall restrict access based on assigned roles.

FR40: The system shall allow viewing all users in a management dashboard.

4. Non-Functional Requirements

4.1 Performance Requirements

NFR1: The system shall respond within 3–5 seconds under normal cloud deployment conditions.
NFR2: The system shall handle at least 100 concurrent users.
NFR3: Reports should generate within 5 seconds.

4.2 Security Requirements

NFR4: Passwords shall be encrypted.
NFR5: The system shall implement JWT authentication.
NFR6: SQL injection protection must be implemented.
NFR7: HTTPS must be used in deployment.
NFR8: JWT tokens shall have expiration and secure storage on client side.

4.3 Reliability

NFR9: System uptime must be 99%.
NFR10: Database backups must be automated and recoverable.

4.4 Usability

NFR11: The UI shall be responsive.
NFR12: The system shall be usable without technical training.

4.5 Maintainability

NFR13: Code must follow clean architecture principles.
NFR14: Unit test coverage should be at least 70%.

NFR15: The system shall follow modular architecture with separation of concerns.

5. External Interface Requirements

5.1 User Interface

- Dashboard
- Product Management Page
- Sales Page
- Supplier Page
- Reports Page
- Login Page
- Sales/Cart Page
- Invoice Preview Page
- User Management Dashboard

5.2 Software Interfaces

- React frontend ↔ ASP.NET REST API
- ASP.NET ↔ MySQL (ADO.NET)
- GitHub Actions ↔ Azure App Service

6. System Architecture

3-Tier Architecture

1. Presentation Layer (React)
2. Business Logic Layer (ASP.NET)
3. Data Layer (MySQL)

- The system implements authentication and authorization as a cross-cutting concern using JWT middleware.

7. CI/CD Requirements

The system shall:

- Automatically build on push to GitHub
- Run unit tests automatically
- Deploy to Azure App Service automatically upon successful build
- Send failure notifications via pipeline logs or alerts

8. Testing Requirements

The system must include:

- Unit Testing (Backend)
- Integration Testing
- Selenium E2E testing
- JMeter Load Testing
- Code coverage reporting
- Authentication testing (valid/invalid login, unauthorized access)
- Sales transaction validation testing
- Inventory consistency testing after transactions

9. Assumptions and Dependencies

- Internet access is available
- Azure services are accessible
- Users have modern web browsers

10. Future Enhancements

- Online customer ordering
- Mobile app
- Barcode scanner integration
- Payment gateway integration